UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"
Câmpus de Presidente Prudente

UNESP - Faculdade de Ciências e Tecnologia
Bacharelado em Ciência da Computação

Yasmin de Oliveira Bonet

Trabalho Prático I
Implementação de uma Árvore B Dinâmica com Acesso a Arquivos Binários

Disciplina: Estruturas de Dados II
Docente: Prof. Danillo Roberto Pereira

Presidente Prudente-SP
2026

Sumário:

2. Decisões de Implementação:.....3
2.1 Estrutura do nó e armazenamento em arquivos:..... 3
2.2 Busca:.....4
2.3 Inserção:..... 4
2.4 Remoção:.....4
2.5 Impressão da Árvore:..... 6
3. Dinâmica do grau mínimo t:..... 7
4. Eficiência:.....9
5. Testes Realizados:.....9
5.1 Teste 1 - Inserção com t=2:..... 9
5.2 Teste 2 - Busca:..... 10
5.3 Teste 3 - Remoção:..... 11
5.4 Teste 4 - Diferentes Valores de t:.....12
5.5 Teste 5 - Casos de Erro e Persistência:.....13
6. Conclusão:.....14

1. Introdução:

Este presente trabalho implementa uma árvore B dinâmica em linguagem C para a disciplina de Estrutura de Dados II. A árvore guarda cada nó em um arquivo binário separado, com o nome gerado durante a execução, e oferece as operações de busca, inserção e remoção seguindo os algoritmos do CLRS. O grau mínimo t é escolhido pelo usuário quando o programa é executado pela primeira vez, e a árvore funciona com qualquer valor de t maior ou igual a 2. O programa também tem uma função que imprime a árvore na tela, nível por nível.

Para utilizar esse tipo de estrutura, escolhi como tema um guia dos episódios do arco do Red John, da série The Mentalist. A chave de cada elemento é calculada como temporada × 100 + episódio, então o episódio 8 da temporada 2 tem a chave 208. Além da chave, cada nó guarda o título, a duração em minutos e a nota do episódio. Foram usados 38 episódios, de T1 a T6E8. O tema não muda o funcionamento da árvore, que continua sendo uma árvore B de chaves inteiras, e serve para testar o código com dados que fazem sentido.

Este relatório explica as decisões tomadas na implementação, como o grau t influencia no formato da árvore e os testes feitos para verificar se a estrutura continua correta depois de inserções e remoções.

2. Decisões de Implementação:

2.1 Estrutura do nó e armazenamento em arquivos:

Cada nó da árvore é uma struct BTreeNode, parecida com a do enunciado, e guarda o número de chaves (n), as chaves (keys), os nomes dos arquivos dos filhos (children) e se o nó é folha (leaf). Como cada chave tem dados do episódio, acrescentei o vetor dados, que anda junto com keys: o episódio de dados[i] pertence à chave keys[i]. Os vetores são alocados com o tamanho que o t pede, ou seja, 2t-1 chaves e 2t filhos por nó.

Cada nó fica em um arquivo próprio, chamado no_0001.bin, no_0002.bin e assim por diante. O nome é gerado por um contador (proximoNumero) que só aumenta, então nunca dois nós recebem o mesmo nome. Os filhos de um nó são guardados como o nome do arquivo deles, e uma string vazia significa que não há filho naquela posição.

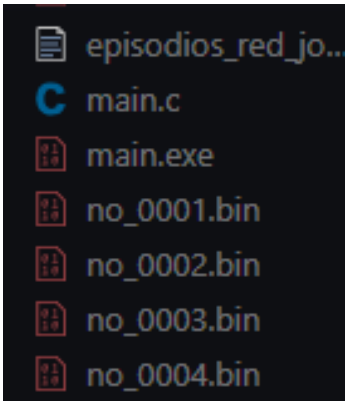


Imagem 1: Exemplos dos nós em cada arquivo.

Para a árvore continuar existindo depois que o programa fecha, existe um arquivo à parte, o controle.bin. Nele ficam três informações: o valor de t, o próximo número de arquivo livre e o nome do arquivo da raiz. Quando o programa abre e encontra esse arquivo, ele reaproveita a árvore e não pergunta o t de novo.

Ao gravar um nó, salvo os campos um por um (n, folha, chaves, episódios e nomes dos filhos), e não a struct inteira com fwrite. A struct tem ponteiros, e um ponteiro gravado no disco perde o sentido na próxima execução, porque o endereço muda. Na hora de ler, o processo é o inverso, e o nó é montado de novo na memória. Só os nós que estão sendo usados ficam carregados, e cada um é liberado (freeNode) assim que deixa de ser necessário.

2.2 Busca:

A busca é recursiva e carrega um nó por vez. Em cada nó, o programa percorre as chaves até achar uma igual ou maior que a procurada. Se achar a chave, devolve o nome do arquivo e a posição dela. Se o nó for folha e a chave não estiver lá, ela não existe na árvore. Caso contrário, o programa libera o nó atual e continua a busca no filho certo. Assim, uma busca lê no máximo um arquivo por nível da árvore.

2.3 Inserção:

Seguindo para outra operação, a inserção no algoritmo do CLRS, que divide os nós cheios antes de descer. Quando o filho por onde a inserção vai passar está cheio (2t-1 chaves), ele é dividido em dois nós com t-1 chaves cada, e a chave do meio sobe para o pai (função dividirFilho). Com isso o pai sempre tem espaço para receber a chave que subiu, e a inserção desce uma vez só, sem precisar voltar. Se a própria raiz estiver cheia, é criada uma raiz nova e a antiga é dividida, e é assim que a árvore ganha um nível. O novo nó criado na divisão ganha um arquivo novo.

Antes de inserir, o programa confere se a chave já existe. Se existir, ele não insere e avisa, já que a árvore não aceita chaves repetidas.

2.4 Remoção:

Também segue o CLRS. A ideia é a mesma da inserção, só que ao contrário: antes de descer para um filho, o programa garante que ele tem pelo menos t chaves, para poder tirar uma chave dele sem quebrar a regra do mínimo. Os casos são:

- **Caso 1:** a chave está numa folha, então basta apagá-la.
- **Caso 2:** a chave está num nó interno. Se o filho da esquerda tem t chaves ou mais, a chave é trocada pelo predecessor, que é a maior chave daquele filho (caso 2a), e o predecessor é removido lá embaixo. Se só o da direita tem de sobra, usa-se o sucessor (caso 2b). Se os dois têm só t-1 chaves, os dois filhos se fundem junto com a chave do pai e a remoção continua no nó fundido (caso 2c).

- **Caso 3:** a chave está mais abaixo, mas o filho por onde se desce tem só t-1 chaves. Se um irmão tem de sobra, o filho pega emprestada uma chave dele, passando pelo pai (caso 3a). Se nenhum irmão tem de sobra, o filho se funde com um irmão (caso 3b).

Quando dois nós se fundem, o arquivo de um deles fica sem uso e é apagado do disco (apagarNo), então o programa não deixa arquivos sobrando. Se a raiz ficar sem chaves e ainda tiver um filho, esse filho vira a nova raiz e a árvore perde um nível. Se a chave não existir, a remoção avisa e a árvore não muda.

```
Arvore existente encontrada (controle.bin).
Grau minimo t = 2

===== Guia do Arco do Red John - Arvore B =====
1. Buscar episodio
2. Inserir episodio
3. Remover episodio
4. Imprimir arvore (por niveis)
5. Carregar episodios do arquivo episodios_pequeno.txt
0. Sair
Escolha: 3
Digite a chave do episodio a remover (ex: 208 para T2E8): 208
Episodio 208 removido.

===== Guia do Arco do Red John - Arvore B =====
1. Buscar episodio
2. Inserir episodio
3. Remover episodio
4. Imprimir arvore (por niveis)
5. Carregar episodios do arquivo episodios_pequeno.txt
0. Sair
Escolha: 4

===== Arvore (nivel por nivel) =====
Nivel 1:                    | 223 |
                          /         \
Nivel 2:       | 111 |                 | 310 |
              /     \                 /       \
Nivel 3:  | 101 |   | 123 |   | 303 | 309 |   | 316 | 320 |
```

Imagem 2: Árvore antes da remoção.

```
===== Guia do Arco do Red John - Arvore B =====
1. Buscar episodio
2. Inserir episodio
3. Remover episodio
4. Imprimir arvore (por niveis)
5. Carregar episodios do arquivo episodios_pequeno.txt
0. Sair
Escolha: 3
Digite a chave do episodio a remover (ex: 208 para T2E8): 310
Episodio 310 removido.

===== Guia do Arco do Red John - Arvore B =====
1. Buscar episodio
2. Inserir episodio
3. Remover episodio
4. Imprimir arvore (por niveis)
5. Carregar episodios do arquivo episodios_pequeno.txt
0. Sair
Escolha: 4

===== Arvore (nivel por nivel) =====
Nivel 1:             | 111 | 223 | 309 |
                    /    /    \     \
Nivel 2:  | 101 |   | 123 |   | 303 |   | 316 | 320 |
```

Imagem 3: Árvore após a remoção.

2.5 Impressão da Árvore:

A função imprimirArvore mostra a árvore no formato do exemplo do enunciado: um rótulo Nivel N: por nível, cada nó entre barras (| 111 | 208 |) e uma linha de /, | e \ ligando cada pai aos filhos. O rótulo está escrito sem acento (Nivel) porque o terminal do Windows nem sempre mostra acentos corretamente. Ela é feita em três passos:

- **Leitura:** a função lê a árvore do disco e monta uma cópia leve dela na memória (NoImp), em que cada nó guarda só o texto que será impresso, por exemplo | 111 | 208 |, e a lista dos filhos.
- **Posições:** primeiro calcula, de baixo para cima, a largura que cada subárvore precisa, que é a soma da largura dos filhos (com um espaço entre eles) ou o texto do próprio nó, o que for maior. Depois, de cima para cima, decide a coluna de cada nó: o pai fica centralizado sobre os filhos, e os filhos ficam lado a lado.
- **Desenho:** os textos e as barras são escritos numa matriz de caracteres, que funciona como uma "tela" em branco. Cada nível ocupa 2 linhas da matriz: a do texto dos nós e a das barras. No fim, a matriz é impressa linha por linha.

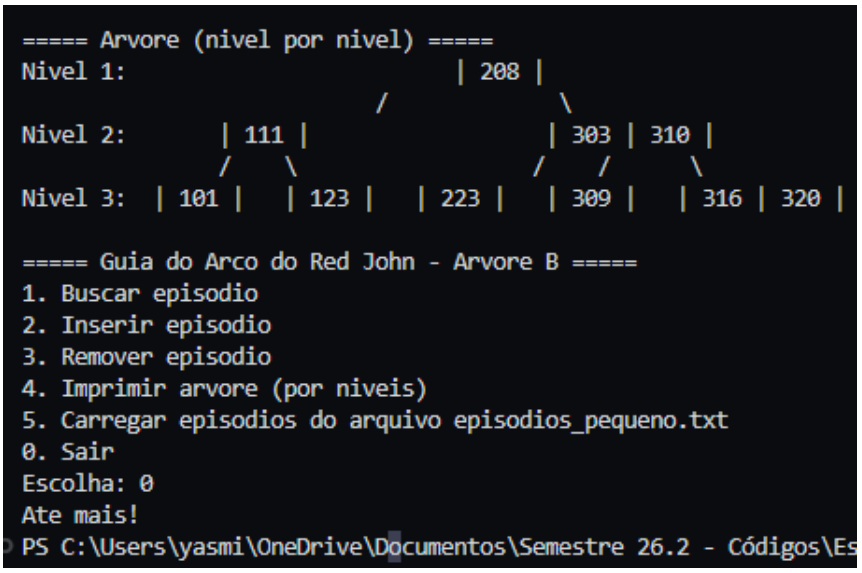


Imagem 4: Impressão da árvore de nível 2.

Como as orientações passadas no trabalho indicam que só é possível imprimir corretamente árvores com t pequeno, o programa verifica antes se o desenho cabe na tela (limite de 118 colunas). Se não couber, como acontece com os 38 episódios, ele imprime só os nós de cada nível, um nível por linha, sem o desenho. Os dados continuam todos corretos, só perdem o centralizado. Uma árvore vazia imprime (arvore vazia).

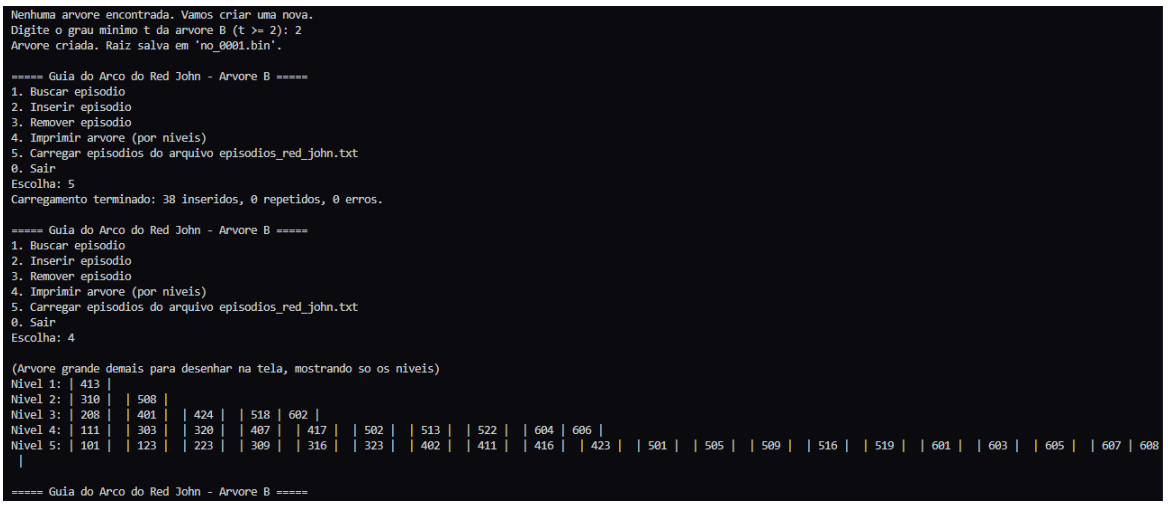


Imagem 5: Impressão da árvore de nível 2 quando o desenho da árvore é muito grande.

3. Dinâmica do grau mínimo t:

O grau mínimo t não está fixo no código. O programa pergunta o valor na primeira execução, recusa qualquer valor menor que 2 e grava o t no controle.bin. Nas próximas execuções, ele é lido de lá, então a árvore sempre é aberta com o mesmo t com que foi criada. Todo o resto depende de t:

- **Tamanho dos nós:** cada nó guarda no mínimo t-1 e no máximo 2t-1 chaves (só a raiz pode ter menos que t-1) e tem de t a 2t filhos, se não for folha. Por isso createNode, salvarNo e carregarNo alocam, gravam e leem os vetores usando o t da árvore, sem nenhum número fixo.
- **Quando um nó divide:** um nó com 2t-1 chaves é considerado cheio. A divisão deixa t-1 chaves em cada metade, o que mantém o mínimo.
- **Quando dois nós se fundem ou emprestam chaves:** a remoção usa t como referência. Um nó com t-1 chaves está no mínimo e não pode perder nenhuma, e um nó com t ou mais chaves tem de sobra.

Na prática, t grande significa nós grandes. Cabem mais chaves por nó, então a árvore fica mais baixa e usa menos arquivos. Com t pequeno os nós são pequenos e a árvore fica mais alta. Para testar isso, carreguei os mesmos 38 episódios com quatro valores de t:

t:	Chaves por nó:	Níveis:	Arquivos de nó:
2	1 a 3	5	35
3	2 a 5	3	17
5	4 a 9	2	8
10	9 a 19	2	4

Tabela 1: Relação de t com chaves, níveis e os arquivos de nó.

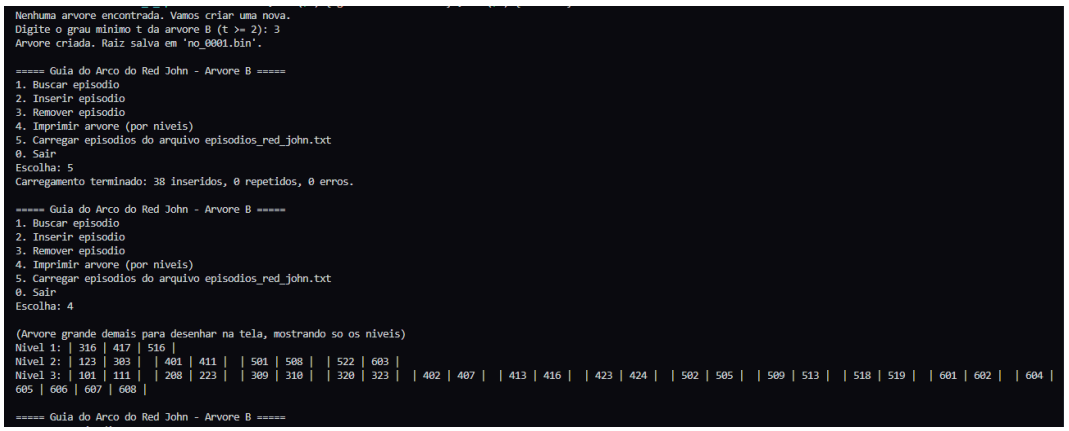


Imagem 6: Impressão da árvore de t=3.

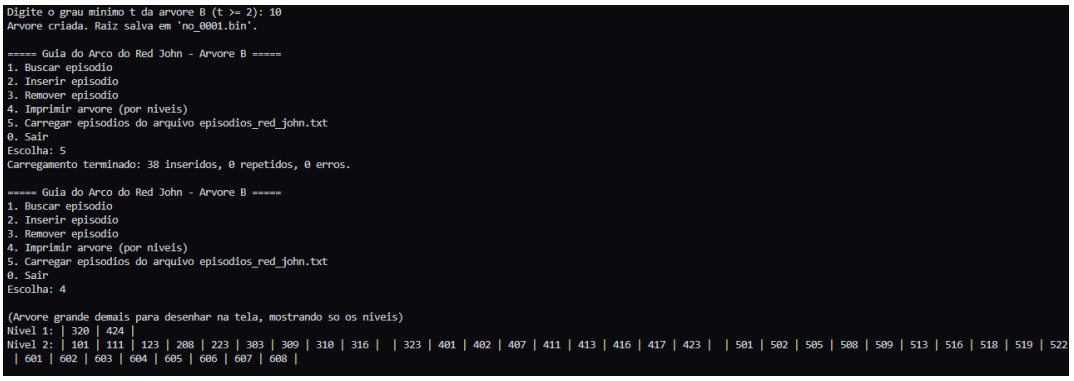


Imagem 6: Impressão da árvore de t=10.

Isso importa porque cada nó é um arquivo, e a busca lê um arquivo por nível. Com t = 2, buscar uma chave exige até 5 leituras de arquivo. Com t = 10, exige no máximo 2. A altura da árvore cresce devagar: com n chaves, ela é no máximo da ordem de log base t de n, então t maior significa árvore mais baixa.

Por outro lado, t grande tem um custo. Cada nó lido do disco é maior, e dentro dele a procura da chave é uma varredura linear, que passa por até 2t-1 chaves. Na prática, o ideal é escolher um t que faça cada nó caber num bloco do disco. Neste trabalho, valores pequenos (2 a 5) facilitam ver a estrutura e imprimir a árvore, e valores maiores (10 ou mais) mostram o ganho de desempenho.

4. Eficiência:

Como cada nó é um arquivo, o custo das operações é medido principalmente pelo número de arquivos lidos, que depende da altura da árvore. Numa árvore B com n chaves e grau mínimo t, a altura é da ordem de log base t de n. A busca lê um arquivo por nível, e a inserção e a remoção descem uma única vez pela árvore (os nós cheios são divididos, e os nós com poucas chaves são preenchidos, antes de descer), então os três custos são proporcionais à altura. Dentro de cada nó a procura é uma varredura de até 2t-1 chaves, já na memória. Antes de inserir e de remover, o programa faz uma busca para saber se a chave existe, o que repete a leitura do caminho, mas não muda a ordem de grandeza. Os resultados de testes realizados

mostram o efeito: com os mesmos 38 episódios, uma busca lê até 5 arquivos com t = 2 e até 2 com t = 10, como apresentado na Tabela 1 da seção anterior.

5. Testes Realizados:

Para verificar se a árvore continua correta depois de inserções, buscas e remoções, fiz cinco testes manuais com o próprio menu do programa. Os testes 1, 2, 3 e 5 usam 10 episódios do arquivo de episódios. O teste 4 usa os 38 episódios completos. Conferi cada resultado com o que se espera de uma árvore B: as chaves em ordem, todas as folhas no mesmo nível e cada nó (menos a raiz) com entre t-1 e 2t-1 chaves.

5.1 Teste 1 - Inserção com t=2:

Criei uma árvore com t = 2 e carreguei 10 episódios pelo menu (chaves 101, 111, 123, 208, 223, 303, 309, 310, 316 e 320). Em seguida imprimi a árvore. O resultado foi:

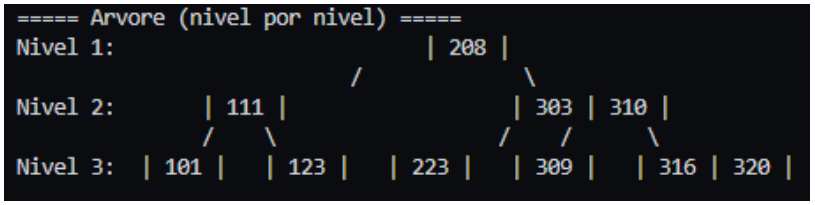


Imagem 7: Impressão da árvore com t=2.

A árvore ficou com 3 níveis, com todas as folhas no mesmo nível. Cada nó tem entre 1 e 3 chaves, que é o permitido para t = 2, e a raiz tem 1 chave. Isso mostra que as divisões de nós cheios aconteceram e mantiveram o balanceamento.

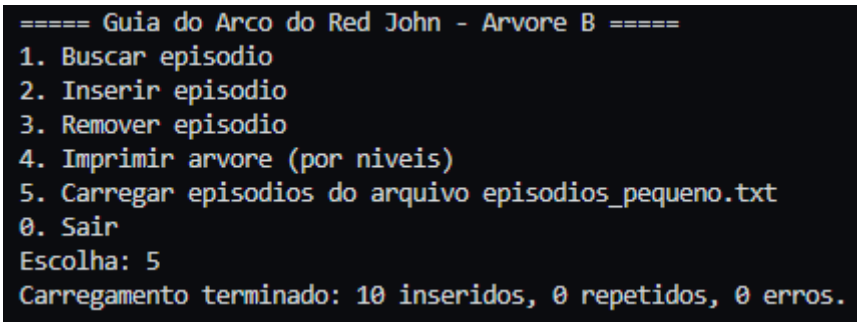


Imagem 8: Carregamento dos episódios.

5.2 Teste 2 - Busca:

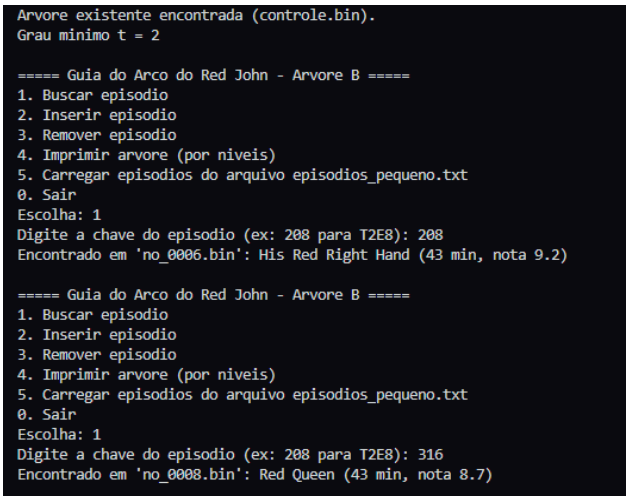
Na mesma árvore, busquei quatro chaves:

Chave Buscada:	Resultado:

208	Encontrada no no_0006.bin (a raiz): Redemption
316	Encontrada no no_0008.bin (uma folha): Red Rover, Red Rover
999	Não encontrada (maior que todas as chaves)
100	Não encontrada (menor que todas as chaves)

Tabela 2: Resultados das chaves encontradas durante os testes

Antes das buscas, fechei o programa e abri de novo. Ele reconheceu a árvore existente (controle.bin, t = 2) e encontrou as chaves, o que confirma que os dados ficam nos arquivos binários. As chaves que não existem não derrubam o programa.



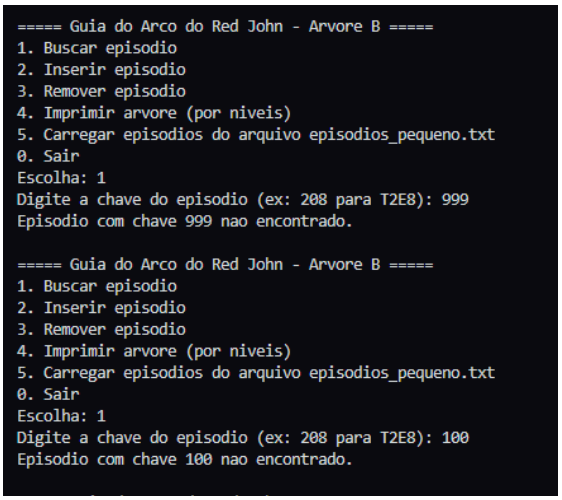


Imagem 9 e 10: Busca dos episódios.

5.3 Teste 3 - Remoção:

Na mesma árvore, removi cinco chaves seguidas, escolhidas para provocar cada caso do algoritmo:

Chave Removida:	Caso do CLRS:	O que aconteceu:
208	2b	A chave estava na raiz. O sucessor, 223, subiu para o lugar dela.
310	3b e 2a	Houve fusão de nós e o predecessor subiu. A raiz ficou vazia e a árvore perdeu um nível (3 para 2).
303	3a	A folha ficou abaixo do mínimo e pegou emprestado do irmão da direita.
111	2c	Os dois filhos tinham o mínimo e se fundiram, com a chave 111 no meio.
309	3a	A folha pegou emprestado do irmão da esquerda.

Tabela 3: Resultados das chaves removidas durante os testes.

A árvore final ficou:

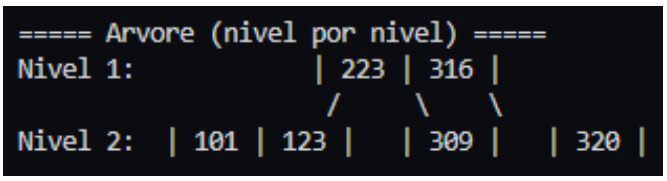


Imagem 11: Resultado da árvore após a remoção de episódios.

Após cada remoção, procurei ver se a árvore continuava válida. Também tentei remover a chave 999, que não existe, e o programa avisou sem alterar nada. Por fim, olhei a pasta: dos 8 arquivos de nó que existiam, sobraram 4 (no_0001, no_0002, no_0004 e no_0008). Os outros foram apagados nas fusões, então o programa não deixa arquivos sem uso.

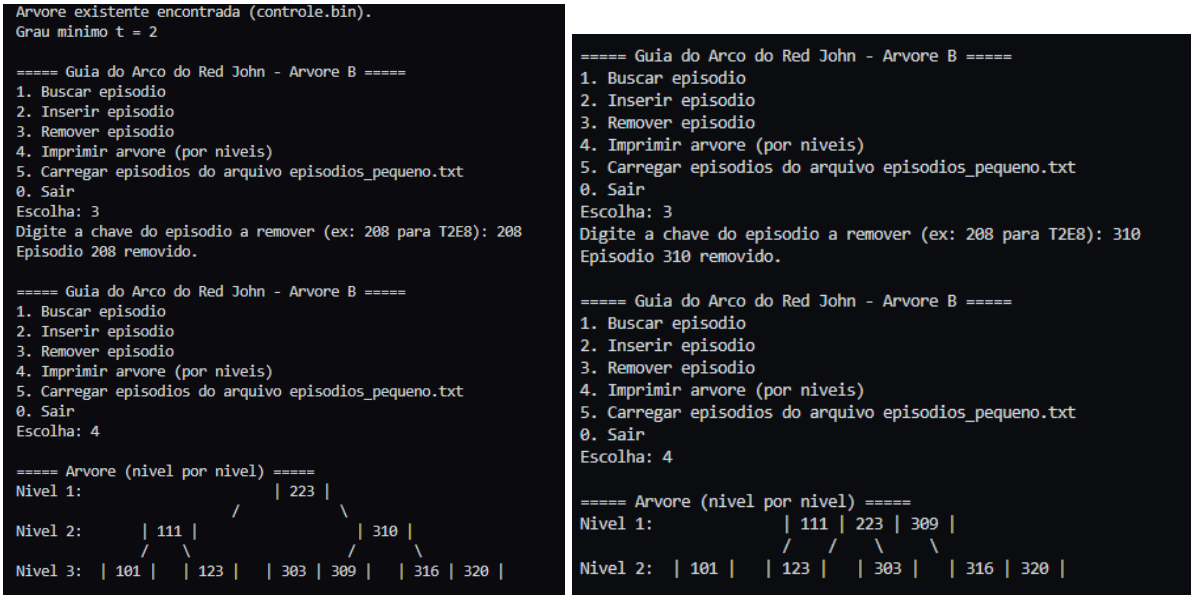


Imagem 12 e13: Mudança de árvore após remoção de episódio.

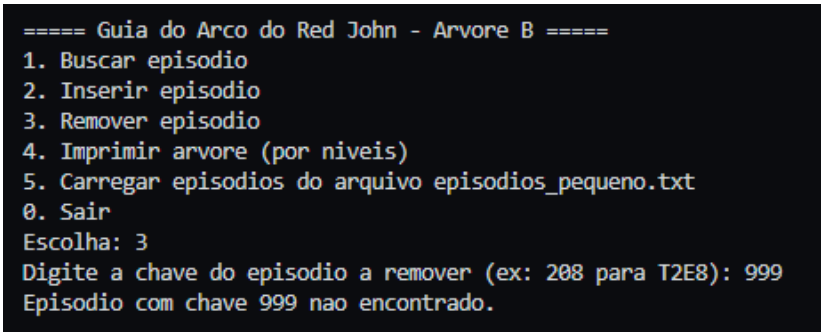


Imagem 14: Remoção de um episódio inexistente.

5.4 Teste 4 - Diferentes Valores de t:

Como comentado anteriormente no trabalho, foi realizado um teste carregando os 38 episódios, mas agora com t = 2, 3, 5 e 10, comparando o formato de cada árvore:

t:	Chaves por nó:	Níveis:	Arquivos de nó:
2	1 a 3	5	35
3	2 a 5	3	17
5	4 a 9	2	8
10	9 a 19	2	4

Tabela 1: Relação de t com chaves, níveis e os arquivos de nó.

Com t maior, a árvore fica mais baixa e usa menos arquivos, como era esperado. Como com os 38 episódios, o desenho final da árvore não cabe na tela, e o programa imprime só os nós de cada nível, um nível por linha, com os dados continuando corretos.

5.5 Teste 5 - Casos de Erro e Persistência:

Exemplos de reação do programa com casos de erros:

- **t inválido:** digitei 1, 0 e abc como grau mínimo. Em todos os casos o programa mostrou “Valor invalido. O grau minimo precisa ser um inteiro >= 2.” e terminou sem criar nenhum arquivo.

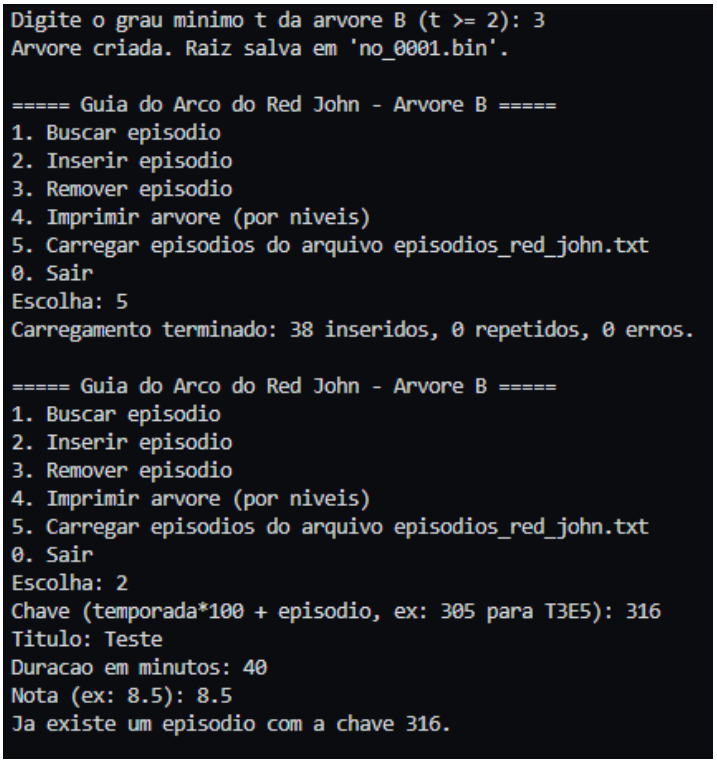
```
Nenhuma arvore encontrada. Vamos criar uma nova.
Digite o grau minimo t da arvore B (t >= 2): 1
Valor invalido. O grau minimo precisa ser um inteiro >= 2.
```

```
Nenhuma arvore encontrada. Vamos criar uma nova.
Digite o grau minimo t da arvore B (t >= 2): 0
Valor invalido. O grau minimo precisa ser um inteiro >= 2.
```

```
Nenhuma arvore encontrada. Vamos criar uma nova.
Digite o grau minimo t da arvore B (t >= 2): abc
Valor invalido. O grau minimo precisa ser um inteiro >= 2.
```

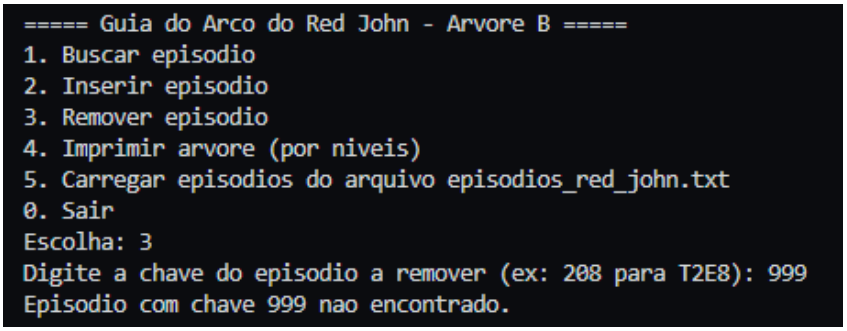
Imagens 15, 16 e 17: Tentativa de t inválido.

- **Chave repetida:** tentei inserir a chave 316, que já existia. O programa avisou “Ja existe um episodio com a chave 316.” e a árvore não mudou.



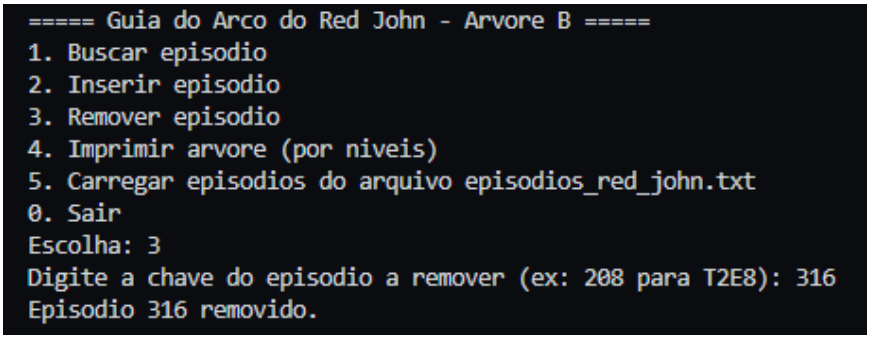
Imagens 18: Inserindo um episódio já existente.

- **Remover chave inexistente:** remover a chave 999 mostrou “Episodio com chave 999 nao encontrado.”.



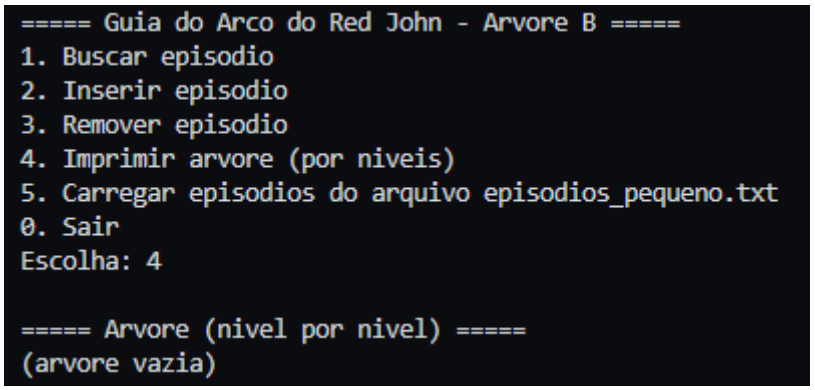
Imagens 19: Removendo chave inexistente.

- **Persistência:** removi a chave 316, fechei o programa e abri de novo. Ele reabriu a árvore sem pedir o t, a busca pela 316 não a encontrou mais e a 417 continuou lá.



Imagens 20: Removendo episódio.

- **Árvore vazia:** removi todas as chaves de uma árvore. A impressão mostrou “(arvore vazia)” e sobrou só um arquivo de nó. Depois inseri a chave 208 e a árvore voltou a funcionar normalmente.



Imagens 21: Impressão de uma árvore vazia.

6. Conclusão:

O trabalho implementou uma árvore B dinâmica em C, com cada nó em um arquivo binário próprio, e com busca, inserção e remoção seguindo os algoritmos do CLRS. O grau mínimo t é escolhido pelo usuário e a árvore funcionou corretamente nos testes com t = 2, 3, 5 e 10. Os testes mostraram que a árvore se mantém balanceada, que os arquivos dos nós que deixam de existir são apagados e que os dados sobrevivem quando o programa é fechado e reaberto. A comparação entre valores de t confirmou que t maior deixa a árvore mais baixa e reduz o número de arquivos lidos.

Como limitações, a impressão só desenha árvores pequenas, o que o próprio enunciado prevê. Para árvores maiores, ela mostra só os níveis. Dentro de cada nó a procura da chave é linear, o que poderia ser trocado por busca binária para valores de t muito grandes. Além disso, as chaves são números inteiros e não podem se repetir.

Fazer este trabalho ajudou a entender por que a árvore B é usada para dados em disco: como cada leitura de arquivo é cara, vale a pena ter nós grandes e uma árvore baixa. Também deu para ver na prática cada caso da remoção, que em sala parece mais difícil do que é.